

Rapport consolidé sur les SGBDR, ACID, les transactions, l'algèbre relationnelle, la normalisation, les statistiques et l'indexation

Résumé exécutif

Le constat principal de ce corpus est stable et robuste. Pour les systèmes transactionnels critiques, le noyau de vérité scientifique demeure le même depuis plusieurs décennies : **modèle relationnel, algèbre relationnelle, optimisation fondée sur les coûts, transactions avec garanties explicites, recovery de type WAL/ARIES, normalisation guidée par les dépendances, et structures d'index ordonnées ou hachées mathématiquement éprouvées**. Les travaux modernes ne remplacent pas ce socle ; ils le prolongent, l'affinent ou tentent d'en corriger certaines limites.

Le premier point de consensus fort est que le paradigme relationnel lancé par Edgar F. Codd chez IBM n'a pas été invalidé par les décennies suivantes. Les systèmes 2024–2026 restent massivement structurés autour des mêmes principes : séparation entre schéma logique et organisation physique, plans d'exécution optimisés à partir de statistiques, et raisonnement fondé sur des équivalences algébriques. Les synthèses récentes sur l'évolution des modèles de données convergent sur ce point : le relationnel ne disparaît pas ; il absorbe et encadre une grande partie des extensions modernes.

Le deuxième point de consensus est que le mot **ACID** n'est utile que s'il est relié à des mécanismes concrets. L'atomicité et la durabilité se rattachent au journal, au commit et au recovery ; l'isolation se rattache aux anomalies effectivement interdites ; la consistance dépend aussi des invariants du schéma et de la logique applicative. En pratique, la vraie question n'est jamais "suis-je ACID ?", mais plutôt "quel niveau d'isolation ai-je réellement, quelles anomalies restent possibles, quel protocole de commit et quel protocole de concurrence sont en jeu, et quelle stratégie de recovery garantit la reprise après panne ?".

Le troisième point de consensus est que les statistiques restent un moteur de premier rang. L'estimation de cardinalité, les histogrammes, les corrélations, les coûts d'I/O et l'énumération des plans sont toujours au centre de la performance. Le front moderne 2020–2026, notamment sur **learned cardinality estimation, learned indexes** et **learned optimizers**, confirme surtout une chose : les limites classiques sont réelles, mais aucun remplacement universel et définitivement supérieur n'est encore établi. Les gains existent, parfois spectaculaires, mais ils sont encore très sensibles au workload, aux distributions, aux mises à jour, au drift et à la robustesse du système.

La doctrine qui se dégage est donc prudente et rigoureuse :

- pour l'**OLTP critique**, conserver le schéma relationnel, la normalisation orientée dépendances, la sérialisabilité ou une isolation explicitement défendue, un moteur WAL robuste, et des B-tree/B+tree comme choix par défaut pour l'ordonné ;
- pour l'**analytique**, déclarer le grain avant toute mesure, distinguer cardinalité conceptuelle et cardinalité d'optimisation, et traiter la summarizability comme une contrainte structurelle, non comme une commodité ;

- pour l'**innovation**, n'adopter learned CE, learned indexes, learned optimizers ou concurrence déterministe qu'après validation empirique complète, avec mesures de robustesse, de dérive et de régression.

Périmètre et méthode

Le périmètre demandé couvre sept blocs étroitement liés : systèmes relationnels, ACID, transactions, algèbre relationnelle, normalisation et dépendances, statistiques/prédictions, et structures de recherche/indexation. Le critère de sélection a donc été double. D'une part, retenir les textes qui ont fixé les définitions, les théorèmes, les modèles de preuve et les bornes asymptotiques encore valides aujourd'hui. D'autre part, retenir des travaux récents 2020–2026 qui confrontent ces bases à la pratique moderne, en particulier sur l'optimisation, les composants appris, le cloud-native, le HTAP, la concurrence déterministe et l'indexation apprise.

Le corpus privilégié repose sur des sources primaires ou quasi primaires : articles fondateurs, revues ACM/IEEE/VLDB/TODS, travaux de laboratoires industriels comme Microsoft Research, publications IBM Research, synthèses académiques récentes, et quelques textes de référence de niveau "book" quand ils servent de formalisation durable. Les sources francophones existent surtout comme exposés pédagogiques, notes de cours ou synthèses ; la littérature primaire du domaine est majoritairement anglophone. C'est donc la littérature scientifique originale qui doit rester la base du raisonnement.

Le livrable téléchargeable contient le rapport complet en Markdown, les diagrammes Mermaid, les tableaux comparatifs, ainsi qu'une bibliographie annotée avec DOI/URL et liens directs vers des PDF ouverts lorsque cela est possible.

Fondements relationnels et lois algébriques

Le point d'ancrage historique reste l'article de 1970 sur le modèle relationnel, prolongé par la complétude relationnelle au début des années 1970, puis par les fondations du calcul et de l'algèbre relationnelle. Sur le plan théorique, l'idée profonde est simple : le résultat d'une requête doit pouvoir être décrit **déclarativement**, sans imposer le chemin d'accès physique. Sur le plan des moteurs, cette idée devient réellement exploitable avec l'école **System R** : on écrit ce que l'on veut, puis l'optimiseur choisit comment l'obtenir.

La conséquence fondamentale est que l'algèbre relationnelle n'est pas seulement un outil pédagogique. C'est le langage des **équivalences sémantiques** utilisées par les optimiseurs. Une même requête peut être transformée en plusieurs expressions équivalentes, puis soumise à une sélection de plan fondée sur les coûts. Les règles les plus importantes à connaître sont celles qui réduisent tôt le volume intermédiaire de données : poussage des sélections, restriction précoce des attributs, et réordonnancement des jointures quand les préconditions sémantiques sont satisfaites.

Noyau des lois à conserver

Le cœur des lois utiles sous **sémantique en ensembles** peut être résumé ainsi :

Famille	Loi-type	Commentaire
Sélection	$\sigma(c1 \wedge c2)(R) = \sigma(c1)(\sigma(c2)(R))$	fusion et commutativité des filtres

Famille	Loi-type	Commentaire
Projection	$\pi(X)(\pi(Y)(R)) = \pi(X)(R)$ si $X \subseteq Y$	idempotence/cascade avec condition d'inclusion
Sélection + projection	$\pi(X)(\sigma(c)(R)) = \sigma(c)(\pi(X \cup \text{attrs}(c))(R))$ sous préconditions	le prédicat doit conserver ses attributs
Jointure	commutativité et associativité usuelles	sous renommage/compatibilité corrects
Sélection + jointure	$\sigma(c)(R \bowtie S)$ peut souvent se pousser sur R ou S	si c ne porte que sur les attributs du côté concerné
Union	commutative, associative, idempotente	vraie en sémantique en ensembles
Différence	non commutative, non associative en général	demande la plus grande prudence

Le point important est que la plupart des raisonnements “naturels” enseignés pour l’algèbre relationnelle supposent des **ensembles**. Or SQL réel fonctionne largement en **bags** ou multiset. Cela change beaucoup de choses. L’idempotence de l’union disparaît avec `UNION ALL`, la projection se comporte différemment lorsqu’elle conserve ou fusionne les duplicats, les réécritures autour de `DISTINCT`, des agrégats, des outer joins et des opérateurs de différence doivent être revalidées formellement. La bonne discipline consiste donc à séparer nettement :

- **algèbre classique** : cadre de preuve propre, très stable ;
- **algèbre en sacs / SQL** : cadre plus proche des moteurs, mais plus délicat ;
- **SQL avec NULL et agrégats** : cadre encore plus subtil, où la logique à trois valeurs et les conventions d’agrégation rendent certaines intuitions trompeuses.

Le consensus établi est que les familles de lois ci-dessus sont indispensables. Ce qui relève encore du front est l’automatisation de la découverte et de la vérification de nouvelles règles de réécriture valides dans des fragments réalistes, notamment en présence de sémantique en sacs.

Transactions, isolation, commit distribué et recovery

Le langage sérieux des transactions ne peut pas se réduire à “ACID”. Le vocabulaire réellement utile exige au minimum de distinguer **sérialisabilité**, **protocole de concurrence**, **protocole de commit distribué**, **anomalies**, **recovery**, et **modèle de visibilité**.

Définitions opératoires

L’**atomicité** signifie qu’une transaction non validée n’a pas d’effet durable partiel. La **durabilité** signifie qu’une transaction validée survit à la panne. La **consistance** signifie que les transitions d’état respectent les contraintes déclaratives et les invariants applicatifs pertinents. L’**isolation** décrit ce qu’une exécution concurrente autorise ou interdit en matière d’interférence. Cette dernière propriété est de loin la plus mal comprise en pratique.

La notion de **sérialisabilité** reste l’étalon de référence. Un historique concurrent est sérialisable s’il est équivalent, selon un critère précis, à une exécution sérielle des mêmes transactions. La version la plus

classique pour les systèmes verrouillés est la **conflict-serializability**, souvent caractérisée à l'aide d'un graphe de précedence. Cette base ne suffit toutefois pas à elle seule pour raisonner sur les systèmes à snapshots, d'où les généralisations plus tardives des niveaux d'isolation.

2PL, phantoms, SSI, 2PC et Calvin

Le **two-phase locking** est un protocole conceptuellement simple : une phase d'acquisition des verrous, puis une phase de relâchement, sans retour à l'acquisition. Sous ses variantes strictes ou rigoureuses, il fournit une base extrêmement solide pour la sérialisabilité et la récupération. Mais il ne résout pas "magiquement" tous les problèmes : les **phantoms** exigent des techniques comme les predicate locks ou les key-range locks.

Le **two-phase commit** est d'une autre nature. Il traite l'**atomic commit distribué**, pas l'isolation. Il faut donc toujours enseigner et documenter 2PL et 2PC séparément. 2PL répond à "dans quel ordre les accès concurrents sont-ils autorisés ?". 2PC répond à "comment plusieurs participants décident-ils ensemble commit ou abort ?". Confondre les deux conduit à des diagnostics faux.

Le **Serializable Snapshot Isolation** est l'un des résultats les plus importants pour la pratique moderne, car il montre qu'il est possible d'obtenir une vraie sérialisabilité sur une base MVCC proche de snapshot isolation, en détectant des motifs dangereux de dépendances, sans revenir purement et simplement au 2PL bloquant sur tout. Cela dit, la qualité pratique ne supprime pas le coût : il faut accepter davantage d'aborts et une sophistication plus grande du moteur.

ARIES demeure l'architecture de recovery historique la plus importante. Elle combine journaux WAL, logique UNDO/REDO, points de reprise, reprise après panne et gestion de granularité fine. Sa longévité n'est pas accidentelle : elle relie proprement la théorie transactionnelle et l'ingénierie des moteurs.

Calvin et la concurrence déterministe constituent un prolongement moderne sérieux. L'idée est de séquencer globalement l'ordre des transactions, puis d'exécuter localement en exploitant cet ordre fixé à l'avance. C'est puissant dans certains environnements distribués à très fort débit, mais les évaluations récentes montrent une dépendance forte au workload, au partitionnement, à la nature des conflits et à la topologie du système. C'est donc une direction importante, pas une doctrine universelle.

Niveaux d'isolation et anomalies

Niveau	Définition synthétique	Garanties principales	Anomalies encore possibles	Implémentations typiques
Read Uncommitted	lectures au plus faible isolement	presque aucune garantie utile	dirty read, non-repeatable read, phantoms, effets dérivés	rare ou limité
Read Committed	pas de lecture sale	interdit les dirty reads	non-repeatable reads, phantoms, write skew selon moteur	verrous courts ou MVCC

Niveau	Définition synthétique	Garanties principales	Anomalies encore possibles	Implémentations typiques
Repeatable Read	même ligne relue de façon stable	interdit dirty + non-repeatable	phantoms selon la mise en œuvre ; pas d'équivalence universelle entre moteurs	2PL ou MVCC renforcé
Snapshot Isolation	photographie cohérente + no dirty read	bonne lisibilité, peu de blocage lecture/écriture	write skew, pas de sérialisabilité générale	MVCC
Serializable	équivalent à un ordre sériel	niveau le plus fort usuel	coût, blocage ou abort plus élevés	strict 2PL, SSI, déterminisme

Le consensus établi porte sur la nécessité d'exprimer les garanties en termes d'**anomalies autorisées/interdites**, et non de simples étiquettes. Le front moderne porte sur le meilleur compromis entre sérialisabilité, débit, latence, contention et distribution.

Diagramme Mermaid des modèles transactionnels

```

flowchart TD
    A[Début transaction] --> B{Contrôle de concurrence}
    B --> C["2PL\nAcquisition de verrous\nPuis libération"]
    B --> D["SSI\nSnapshot + détection\ndépendances dangereuses"]
    A --> E{Transaction distribuée ?}
    E --> F["2PC\nPrepare\nVote\nCommit ou Abort"]
    A --> G{Ordonnancement prédéfini ?}
    G --> H["Calvin\nSéquençage global\nExécution locale"]
    A --> I["WAL / ARIES\nUNDO / REDO / restart"]

```

Dépendances, normalisation, décomposition, cardinalité et granularité

La théorie de la normalisation n'est pas une liste de slogans, mais une théorie des **dépendances** et des **décompositions**. Les dépendances fonctionnelles structurent la 3NF et la BCNF ; les dépendances multivaluées structurent la 4NF ; la DKNF pousse l'idéal encore plus loin.

Dépendances fonctionnelles et règles d'inférence

Le noyau formel minimal à conserver comprend les **axiomes d'Armstrong** :

- réflexivité ;
- augmentation ;
- transitivité.

À partir d'eux, on dérive des règles utiles comme l'union, la décomposition et la pseudo-transitivité. Pour un schéma donné, le calcul de la **fermeture** d'un ensemble d'attributs et de la **fermeture** d'un ensemble de dépendances reste l'outil de base pour raisonner sur les clés, la redondance et les décompositions.

3NF, BCNF, 4NF, DKNF

La **3NF** permet des schémas corrects tout en préservant mieux les dépendances. La **BCNF** est plus stricte : toute dépendance non triviale doit avoir une superclé à gauche. En pratique, elle élimine davantage d'anomalies, mais peut sacrifier la préservation des dépendances. C'est pourquoi la 3NF synthétisée à partir des FDs garde sa pertinence pratique.

La **4NF** devient nécessaire quand il existe de vraies dépendances multivaluées indépendantes. Le théorème important ici est l'existence d'une **décomposition sans perte** vers des schémas 4NF. La **DKNF** constitue un idéal conceptuel : toutes les contraintes sont réductibles aux domaines et aux clés. Elle a une grande valeur doctrinale, mais elle est rarement une cible opérationnelle universelle en production.

Décomposition sans perte et préservation des dépendances

Pour une décomposition binaire de R en R_1 et R_2 , un critère classique de **jointure sans perte** est que l'intersection $R_1 \cap R_2$ détermine au moins l'un des deux fragments dans la fermeture F^+ . C'est un critère central, car une décomposition parfaitement "propre" sur le papier mais avec jointure avec perte détruit le sens du schéma.

La **préservation des dépendances** est un critère différent : après décomposition, peut-on encore faire vérifier les dépendances pertinentes sans recomposer globalement toute la relation ? C'est ici qu'apparaît l'un des arbitrages les plus importants de la théorie : **BCNF** peut être mieux normalisée mais perdre la préservation de certaines dépendances ; **3NF** conserve souvent mieux les contraintes de validation locale.

Cardinalité et granularité

Dans votre périmètre, il faut absolument distinguer deux sens du mot **cardinalité**.

Le premier sens relève du **modèle conceptuel** : cardinalités min/max des associations, participation obligatoire ou optionnelle, unicité, multiplicité. C'est de la sémantique métier.

Le second sens relève de l'**optimisation** : cardinalité estimée à la sortie d'un opérateur, d'une jointure, d'un filtre, d'une agrégation. C'est de la statistique et de la planification.

La **granularité**, elle, devient décisive côté analytique. Le "grain" d'une table de faits fixe ce qu'une ligne représente exactement. Sans grain rigoureusement défini, les agrégations, comparaisons temporelles et ratios deviennent fragiles. La **summarizability** n'est donc pas un confort, mais une condition de validité analytique.

Statistiques, estimation, poids, prévision et déterminisme

L'estimation de cardinalité demeure l'un des problèmes centraux de l'optimisation. Le grand enseignement est que les meilleurs moteurs ne se contentent pas "d'exécuter vite" : ils **prédissent** des tailles intermédiaires, **évaluent** des coûts, **pondèrent** des alternatives et **choisissent** sous incertitude.

Histogrammes et estimation de cardinalité

Les histogrammes restent une base extraordinairement solide. Ils modélisent des distributions de valeurs de façon compacte et permettent des estimations de sélectivité raisonnables à coût faible. Les limites sont connues : corrélations inter-attributs, distributions très non uniformes, forte dynamique des données, jointures complexes et effets cumulatifs d'erreurs.

La grande synthèse récente sur l'optimisation rappelle que trois éléments restent dominants :

- **erreur de cardinalité ;**
- **modèle de coût imparfait ;**
- **espace de plans gigantesque.**

La rétrospective 2025 sur la qualité réelle des optimiseurs confirme que les mauvaises estimations de cardinalité figurent toujours parmi les causes majeures des plans désastreux.

Learned cardinality estimation

Le front 2020–2026 sur les estimateurs appris est scientifiquement sérieux. Les évaluations approfondies montrent toutefois une image nuancée.

Le **consensus de frontière** actuellement le plus crédible est le suivant :

- des gains réels sont possibles par rapport aux modèles strictement histogrammés ;
- aucun estimateur appris unique ne domine toutes les distributions, tous les schémas et tous les workloads ;
- les coûts de maintenance, de ré-entraînement, de dérive et parfois de sécurité ne sont pas accessoires ;
- l'adaptation automatique du meilleur estimateur selon le contexte est elle-même un sujet de recherche important.

Les travaux récents sur les attaques par empoisonnement de l'historique de requêtes sont particulièrement importants pour un jugement rigoureux : ils montrent que les estimateurs appris ne posent pas seulement une question de précision, mais aussi de **robustesse systémique**.

Prévision, poids et modèles déterministes ou stochastiques

Les travaux de prévision montrent une leçon généralisable aux systèmes de données : les meilleurs résultats viennent souvent de **combinaisons** de modèles, de pondérations robustes et d'évaluations empiriques, bien plus que d'un "modèle miracle". Les compétitions de prévision ont rappelé que les approches statistiques bien conçues restent extrêmement compétitives face aux méthodes plus complexes.

Dans votre cadre, le mot **poids** a plusieurs sens utiles :

- poids dans une fonction de coût d'optimisation ;
- poids implicites d'un accès plus fréquent dans un index ou un arbre de recherche ;
- poids d'une série dans une combinaison de prévisions ;
- poids d'observations dans une estimation statistique robuste.

La distinction **déterministe vs stochastique** doit rester explicite. Un protocole transactionnel déterministe n'est pas la même chose qu'un modèle statistique déterministe. De même, un estimateur appris "probabiliste" n'implique pas un moteur transactionnel non déterministe. Ce sont des axes différents du raisonnement.

Structures de recherche, index et parcours

Le socle des structures d'accès reste remarquablement stable : B-tree/B+tree pour l'ordonné, hachage dynamique pour l'égalité, analyses pondérées pour certains problèmes de recherche, et parcours linéaires de graphes pour de nombreuses tâches algorithmiques.

B-tree, B+tree, hachage et optimal BST

Le **B-tree** et, dans la pratique moteur, le **B+tree**, restent les structures de référence pour l'index ordonné. Leur intérêt profond est double : coût logarithmique en fonction de la hauteur, et excellente adéquation au stockage paginé. Les scans de plages, les recherches exactes ordonnées et les parcours séquentiels y sont naturels. En concurrence, ces structures ont aussi bénéficié d'un très grand nombre de raffinements de verrouillage et de recovery.

L'**optimal binary search tree** appartient davantage au versant théorique et aux recherches pondérées. Il répond à la question suivante : si les clés n'ont pas les mêmes probabilités d'accès, quel arbre minimise la longueur de chemin espérée ? C'est un outil important pour raisonner sur les **poids d'accès**, pas un remplaçant direct du B+tree transactionnel paginé.

L'**extendible hashing** et le **linear hashing** sont les grandes réponses dynamiques aux limites du hachage statique. Ils offrent de très bonnes performances en égalité, mais supportent mal les recherches par intervalle et les parcours ordonnés. Le choix entre arbre ordonné et index haché n'est donc pas esthétique ; il dépend du type de prédicat.

Parcours DFS et BFS

Les algorithmes de parcours **DFS** et **BFS** ne sont pas des index, mais ils appartiennent à votre périmètre parce qu'ils fixent des bornes et des schémas de calcul encore omniprésents dans les moteurs, les analyseurs de plans, les graphes de dépendances et les preuves de correction :

- **DFS** : utile pour composantes connexes, cycles, ordres topologiques et nombreux algorithmes linéaires ;
- **BFS** : utile pour distances minimales en graphes non pondérés et explorations par couches.

Leur complexité canonique en représentation par listes d'adjacence est $O(V + E)$. C'est une borne fondamentale et très stable.

Learned indexes

Les **learned indexes** reformulent un index comme une approximation du mapping clé → position. Le message scientifique 2018 a été très influent : si la distribution des clés peut être apprise, alors une partie du rôle de l'index peut être jouée par un modèle. Cela a ouvert un front très actif.

Mais la meilleure lecture 2025–2026 est prudente. Les synthèses expérimentales et les bornes théoriques récentes montrent que :

- les gains dépendent fortement du domaine des clés, de la dimension, de la distribution, du taux de mise à jour et du matériel ;
- en multi-dimensionnel, les gains sont loin d'être uniformes ;
- les contraintes théoriques de complexité ne disparaissent pas parce qu'un index est "learned" ;
- la prise en compte explicite du **workload** et des **poids d'accès** peut changer radicalement la qualité du résultat.

Comparaison des structures d'index

Structure	Recherche	Insertion / suppression	Coût de mise à jour	Support des plages	Convivialité concurrence
B-tree / B+tree	$O(\log_B N)$	$O(\log_B N)$ + splits/merges	régulier, bien maîtrisé	excellent	très bonne
Optimal BST	dépend des poids ; objectif de coût moyen minimal	reconstruction coûteuse si distribution change	sensible aux changements de fréquence	correct mais secondaire	faible intérêt direct moteur
Extendible hashing	proche de $O(1)$ attendu	scissions localisées + croissance du directory	bon si égalité dominante	mauvais	correct
Linear hashing	proche de $O(1)$ attendu	croissance progressive	amorti intéressant	mauvais	correct
Hash index classique	$O(1)$ attendu	bon si peu de dérive	simple pour égalité	très mauvais	acceptable
Learned index	dépend du modèle, du fallback et des bornes	potentiellement coûteux avec drift	sensible au retraining	variable selon design	encore très dépendant de l'implémentation

Diagramme Mermaid des familles d'index

```
graph TD
  A[Structures de recherche] --> B[Arbres ordonnés]
  A --> C[Hachage]
  A --> D[Indexés appris]
  B --> E[BST]
  B --> F[B-tree]
  B --> G[B+tree]
  C --> H[Hash index]
```

```

C --> I[Extendible hashing]
C --> J[Linear hashing]
D --> K[Learned index 1D]
D --> L[Learned index multi-dimensionnel]

```

Cross-check et notes critiques

Le travail de cross-check conduit à plusieurs jugements nets.

Le premier est que les **sources canoniques** et les **synthèses récentes** ne sont pas en conflit sur l'essentiel. Les travaux 2024–2026 sur cloud-native databases, HTAP, optimiseurs ou learned components confirment surtout la persistance du socle relationnel, transactionnel et algébrique.

Le deuxième est que plusieurs affirmations modernes demandent une qualification explicite.

Revendication moderne	Verdict après cross-check	Statut
"Les learned CE remplacent les histogrammes"	Faux en général. Gains réels, mais robustesse et domination universelle non établies.	Front de recherche
"Les learned indexes remplacent les B+trees"	Faux en général. Très bons résultats sur certains workloads, mais pas de consensus universel.	Front de recherche
"Snapshot Isolation est pratiquement sérialisable"	Faux en général. Le write skew et d'autres motifs restent possibles.	Consensus établi contre cette affirmation
"Le déterminisme est meilleur que 2PL/SSI"	Faux en général. Très contextuel, surtout intéressant en distribution ciblée.	Front de recherche
"BCNF est toujours préférable à 3NF"	Faux en général. BCNF peut perdre la préservation des dépendances.	Consensus établi
"La normalisation améliore toujours la performance"	Faux en général. Bonne pour la cohérence OLTP ; compromis différents en analytique et en lecture lourde.	Consensus établi

Le troisième jugement est méthodologique : dès qu'un résultat moderne annonce des gains importants, il faut vérifier au minimum cinq points avant d'en tirer une doctrine :

- quelle distribution de données a été utilisée ;
- quel est le taux de mise à jour et le coût de maintenance ;
- quelle métrique d'erreur ou de performance est retenue ;
- quelles baselines classiques ont été choisies ;
- quelle est la robustesse face au drift, à l'adversarial et au changement de workload.

Questions ouvertes et limites

La première limite est documentaire : une partie des standards SQL formels reste plus difficile d'accès libre que les grands articles académiques qui les critiquent ou les clarifient. Pour certains sujets, la

meilleure source libre est donc une critique scientifique de haute qualité plutôt que le texte normatif complet.

La deuxième limite est sémantique : il n'existe pas une unique "liste fermée" des lois de l'algèbre relationnelle applicable sans nuance à l'ensemble de SQL. Dès qu'on ajoute sémantique en sacs, `NULL`, agrégats, fenêtres, outer joins ou opérateurs non monotones, le catalogue des équivalences doit être re-spécifié.

La troisième limite est épistémique : les learned CE, learned indexes et learned optimizers sont désormais des sujets sérieux, mais ils n'ont pas encore le même statut que les piliers historiques du domaine. Leur adoption doit rester subordonnée à la preuve empirique, à la stabilité opérationnelle et à la compréhension de leurs bornes théoriques.

Livrable téléchargeable

Le rapport complet en Markdown, avec tableaux, diagrammes Mermaid et bibliographie annotée incluant DOI/URL et liens vers des PDF ouverts lorsque disponibles, est téléchargeable ici :

[rapport_sgbdr_doctrine_2026.md](#)
